

L'intelligence artificielle générative

Partie 10 : du RAG au Agents



Présenté par **Morgan Gautherot**



Contexte et enjeux

Les promesses

- Compéhension naturelle de tous types de documents
- Réponses précises extraites directement de vos données
- Déploiement simple et uniforme
- Adaptation automatique à vos contenus spécifiques

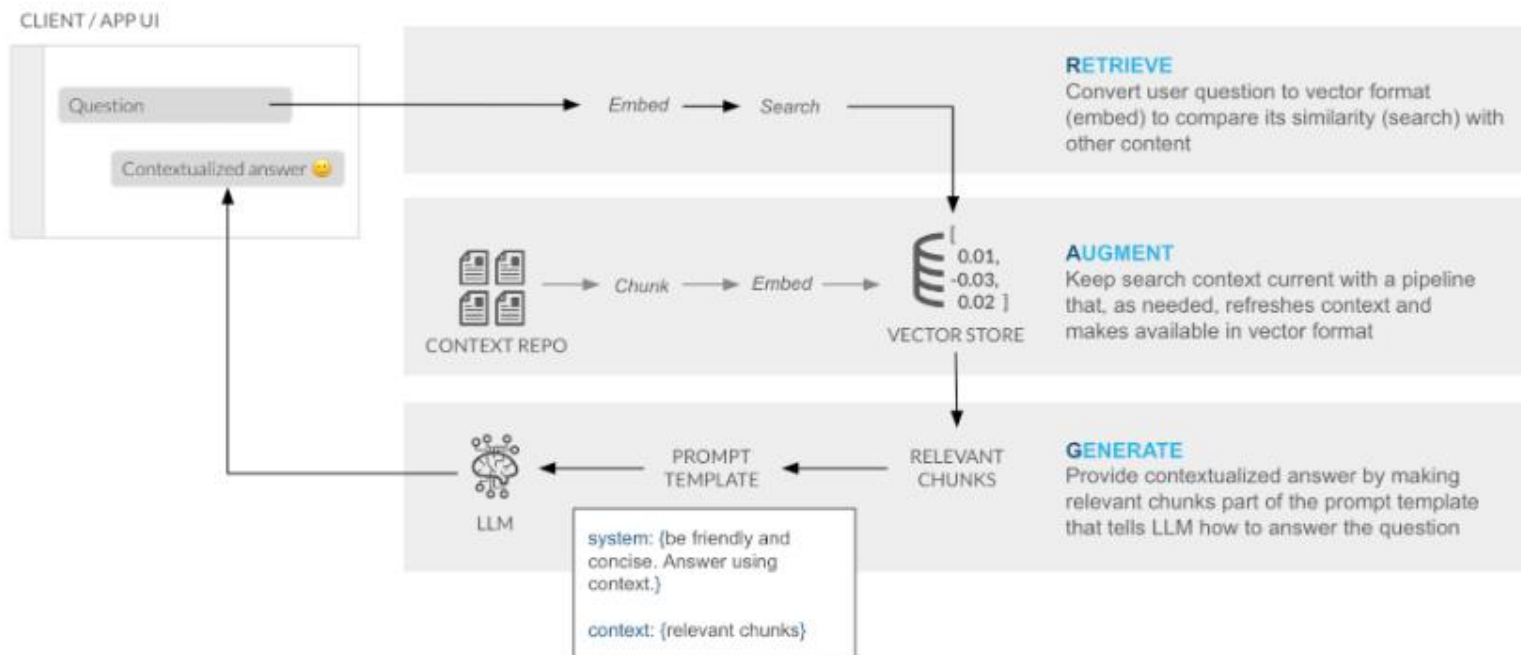
La réalité après le passage en production

- Imprécisions fréquentes sur les données complexes
- Difficultés avec certains formats (schéma industriels, tableaux techniques).
- Incohérences entre versions de documents techniques
- Raisonnement limité sur des données interdépendantes

Un écart significatif entre promesse théorique et réalité pratique, particulièrement critique

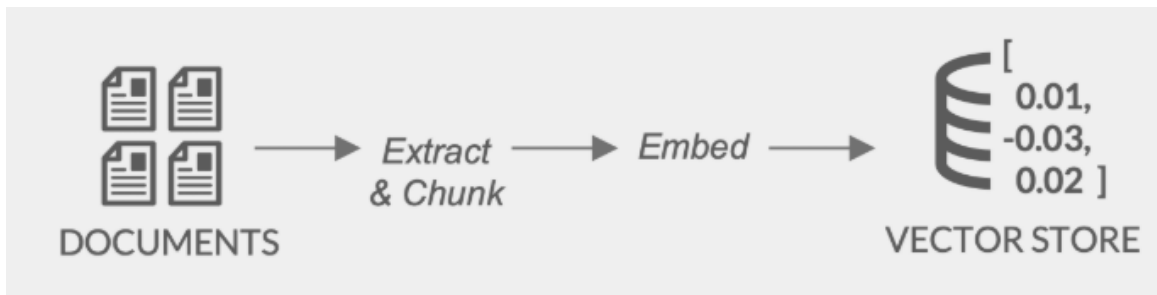


Utilisez la base vectorielle pour répondre à l'utilisation





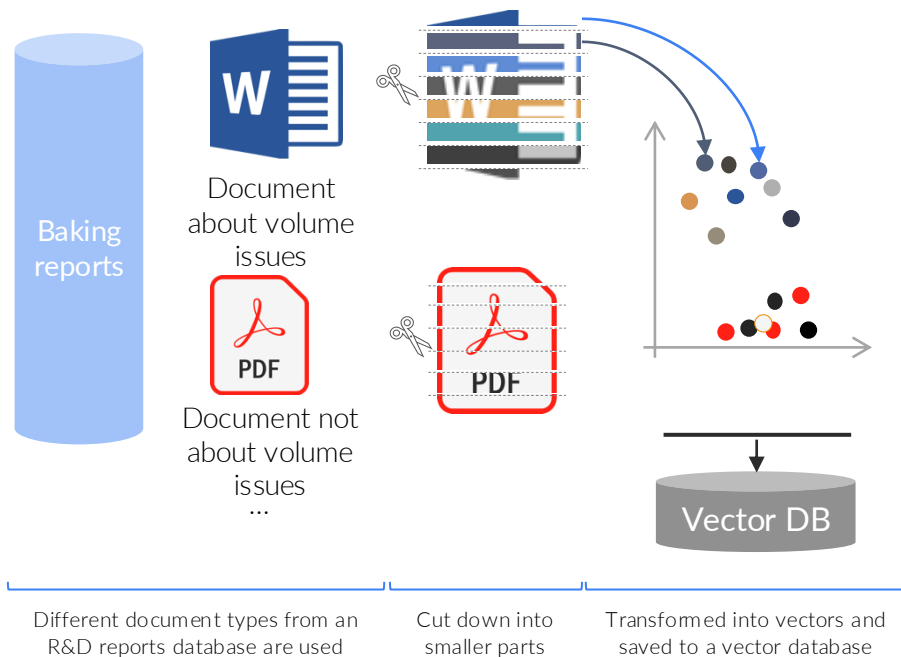
Créer une base vectorielle





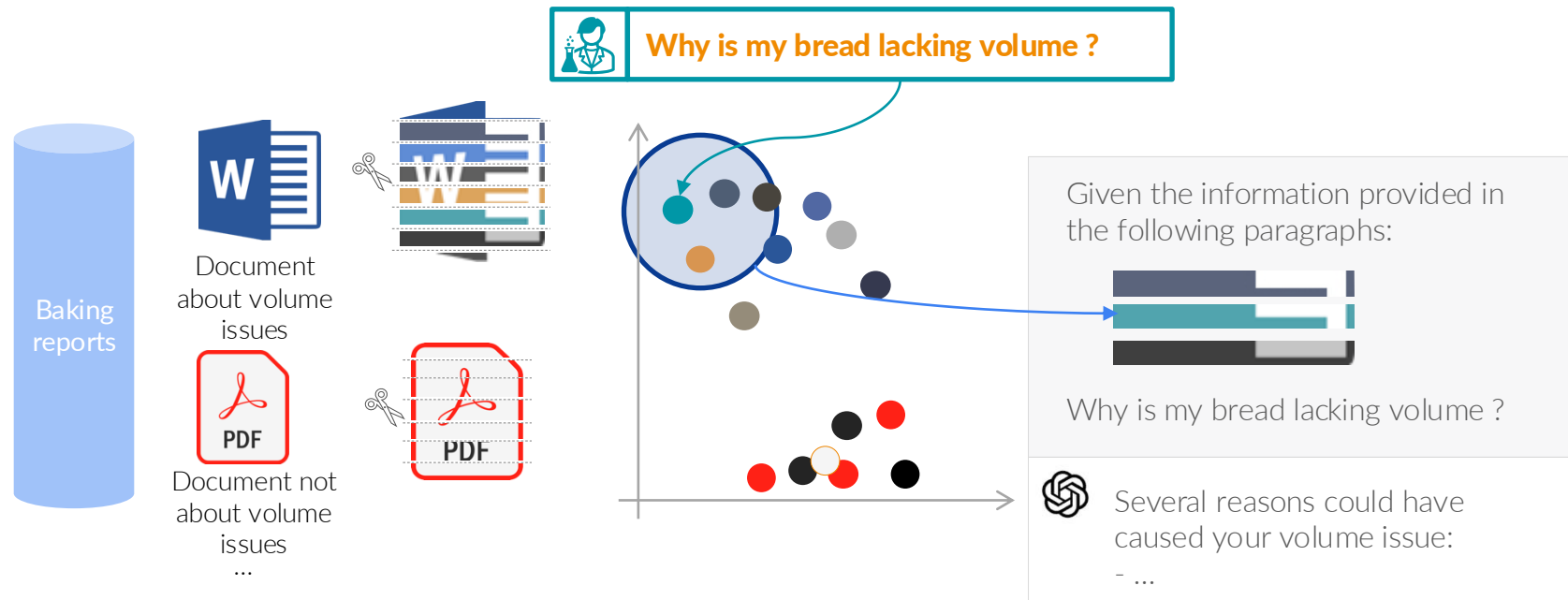
What is Retrieval Augmented generation ?

Retrieval Augmented Generation (RAG) uses the power of Large Language Models to interrogate custom database of documents





What is Retrieval Augmented generation ?



Different document types from a breadmaking troubleshooting reports database are used

Cut down into smaller parts

Embed the user query
Select the vectors closest to the query

Given context from relevant breadmaking troubleshooting reports, we ask a Large Language Model to answer our question



Limites des RAGs

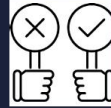
- Peut couter cher
- Peut-être long pour obtenir une réponse
- La performance est compliquée à calculer
- Gestion documentaire

Chunking - Un défi sous-estimé



Taille

Inconvénients



Avantages



Problématiques sous estimées

Petite (256 tokens)

- Perte de contexte,
- Fragmentation du sens
- Multiplication des chunks

- Précision accrue,
- Économie de tokens
- Retrieval ciblé

Moyen (512-1024)

- Compromis sans excellence

- Equilibre relatif
- Standard industriel

Grands (2048)

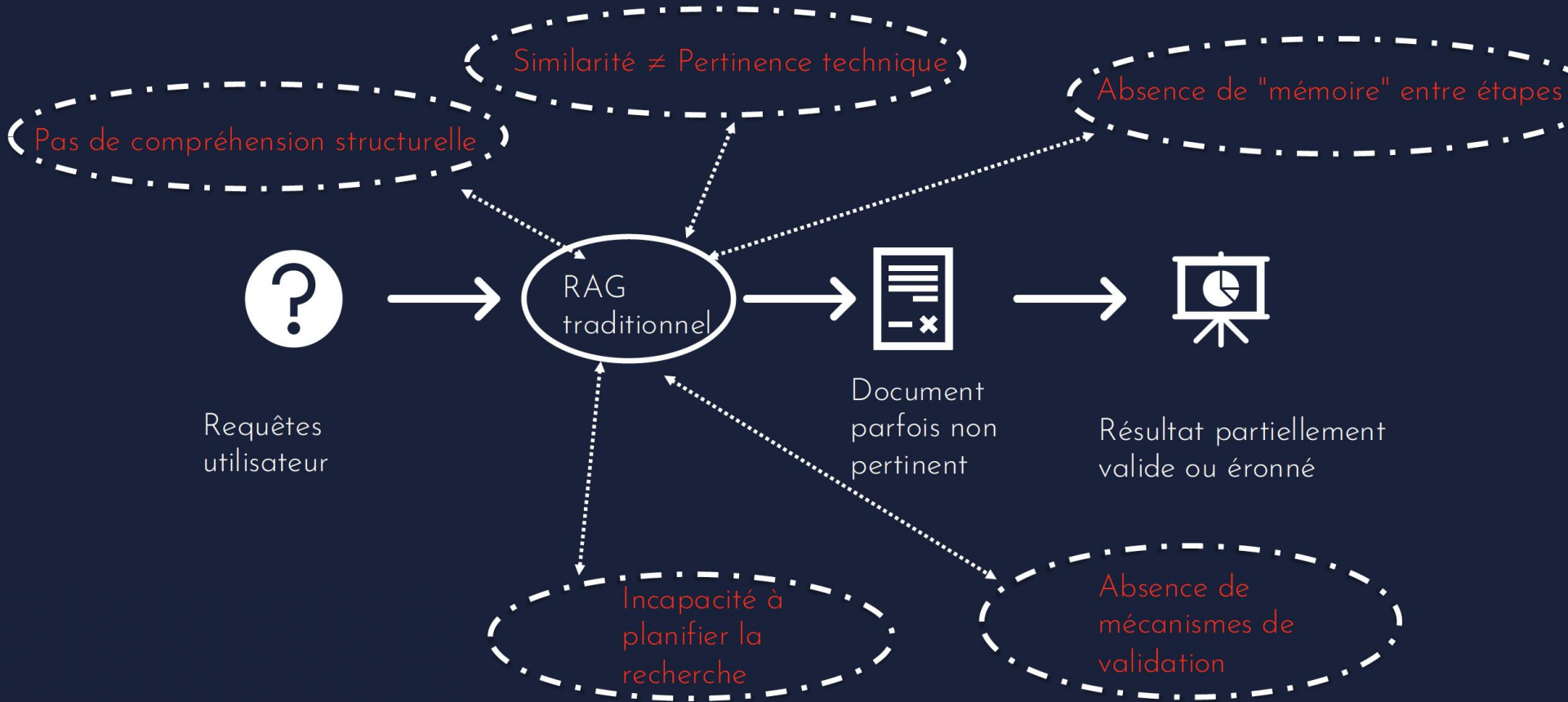
- Coût élevé,
- Dilution du signal
- Taille context window

- Préservation du contexte,
- Cohérence

- Chevauchement: nécessaire mais coûteux (+30-50% de stockage)
- Préservation de la structure: sections, paragraphes, hiérarchie
- Extraction d'éléments tabulaires techniques et structurés
- Méta-information critique (unités, tolérances, références normatives)

→ Un paramètre crucial souvent configuré par défaut, pourtant déterminant pour la qualité des réponses

Causes profondes des échecs : Limitations architecturales



Solutions partielles développées

HyDE: Hypothetical Document Embeddings

- Génération d'hypothèses de réponse
- ✓ Meilleure formulation des requêtes
- ✗ Risque d'hallucination dans l'hypothèse

Rerankers: affiner la pertinence

- Cross-encoders, BCE
- ✓ Amélioration de 2-3x du classement
- ✗ Coût computationnel élevé

Métriques d'amélioration:

- RAGAS: +10-15% sur la fidélité technique
- ROUGE: +5-10% sur la complétude
- Satisfaction utilisateurs techniques: +15-20%

RAG récursif et multi-étapes

- Raffiner les requêtes progressivement
- ✓ Exploration plus complète des spécifications techniques
- ✗ Latence accrue

ColBERT et architectures late-interaction

- Interaction token-level fine
- ✓ Matching précis et explicable
- ✗ Scaling limité

Limites d'une architecture généraliste -> Vers une spécialisation

Architecture toujours monolithique

- Même pipeline pour tous types de documents techniques
- Pas d'adaptation aux spécificités des schémas industriels
- Un seul LLM pour toutes les tâches techniques

Absence de spécialisation par contenu

- Traitement identique des textes, schémas P&ID, diagrammes électriques
- Embeddings génériques pour formats techniques spécifiques
- Pas d'extracteurs dédiés aux formats industriels spécialisés



Nécessité de spécialiser les RAG

Limites de contextualisation persistantes

Pas de planification stratégique

- Améliorations tactiques sans vision stratégique
- Réactif plutôt que proactif
- Incapacité à décomposer les problèmes techniques complexes

Impossibilité de traiter certains formats industriels

Du monolithique au modulaire.

Repenser fondamentalement l'approche

Caractéristique	RAG Traditionnel	Approche Agents
Architecture système	Système unique "one-size-fits-all"	Équipe technique spécialisée avec compétences complémentaires
Structure de traitement	Pipeline linéaire rigide	Architecture flexible et modulaire
Adaptabilité	Optimisé pour cas généraux	Adaptable aux cas techniques spécifiques
Gestion documentaire	Traitement homogène	Traitement spécialisé par type de documentation technique

Capacités essentielles des agents



Construire des agents : les frameworks

Framework	Paradigme Principal	Force Principale	Idéal Pour
LangGraph	Workflow de prompts basé sur des graphes	Contrôle DAG explicite, branchement, débogage	Tâches complexes multi-étapes avec gestion d'erreurs avancée
OpenAI Agents SDK	Toolchain OpenAI de haut niveau	Outils intégrés comme recherche web et fichiers	Équipes utilisant l'écosystème OpenAI avec support officiel
Smolagents	Agent minimaliste centré sur le code	Configuration simple, exécution directe	Automatisation rapide sans surcharge
CrewAI	Collaboration multi-agents (équipes)	Workflows parallèles basés sur des rôles, mémoire partagée	Tâches complexes nécessitant plusieurs spécialistes
AutoGen	Chat multi-agents asynchrone	Conversations en direct, piloté par événements	Scénarios nécessitant concurrence et voix multiples
Semantic Kernel	Intégrations d'entreprise basées sur des compétences	Multi-langues, conformité d'entreprise	Environnements .NET, grandes organisations, orchestration robuste
LlamaIndex Agents	RAG avec indexation intégrée	Synergie retrieval + agent	Cas d'usage centrés sur la recherche et fusion de données
LLPhant	Framework PHP pour l'intégration de LLMs	Compatibilité Laravel, simplicité d'intégration	Applications web PHP nécessitant des capacités IA modernes



Analysez l'impact des nouvelles normes d'efficacité énergétique sur les performances de notre centrale à cycle combiné, et déterminez les modifications nécessaires pour optimiser le rendement tout en respectant les limites d'émissions NO_x. »



1. Requête utilisateur → Agent orchestrateur

2. Décomposition et planification

T1: Identifier normes énergétiques applicables

T2: Extraire performances actuelles du cycle combiné

T3: Analyser paramètres d'optimisation disponibles

T4: Modéliser impact des modifications sur les valeurs

T5: Synthétiser recommandations techniques

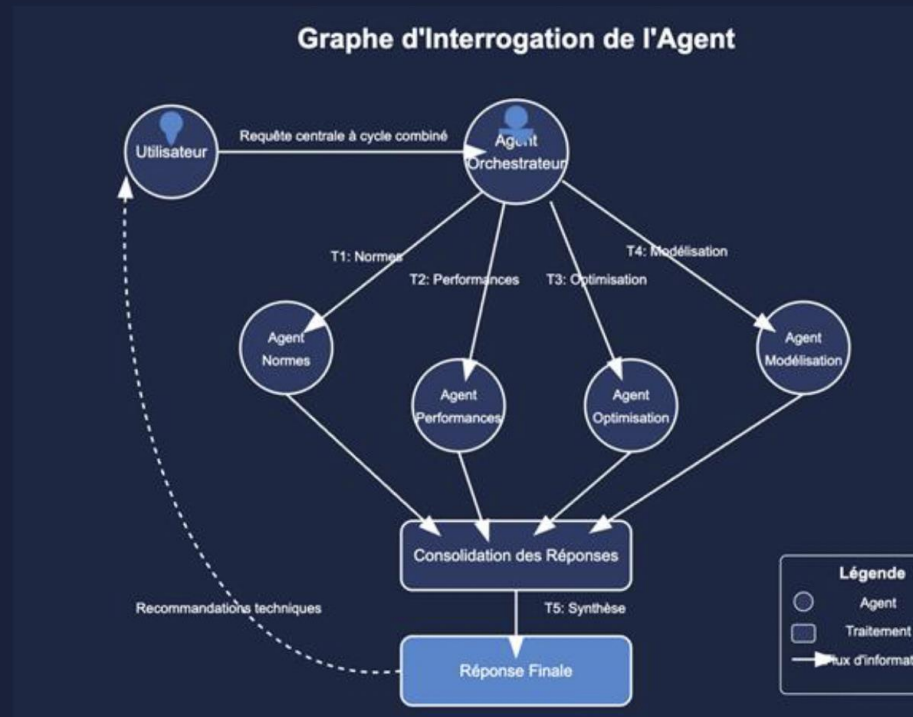


3. Dispatch vers agents spécialisés



4. Exécution parallèle et coordination

5. Consolidation et génération de réponses



Synthèse d'exemple d'améliorations

Métrique	RAG standard	Approche multi-Agents
Précision technique	61%	94%
Rappel (complétude)	58%	92%
F1 score global	59%	93%
Temps de traitement	Base	35/45%
Taux de requêtes techniques résolues	42%	89%
Satisfaction des utilisateurs techniques	5,4/10	8,7/10
ROI documenté	Variable	3,5 à 7,2 X l'investissement

Domaines d'excellence d'utilisation des agentic RAG

1. Documentation technique complexe multi-formats
2. Analyse de performance et optimisation
3. Conformité réglementaire et sécurité
4. Diagnostic et résolution de problèmes

Avantages de l'approche multi-agents



Précision et fiabilité technique accrues

- Extraction exacte des spécifications techniques
- Compréhension des schémas et diagrammes industriels
- Capacité à gérer les systèmes complexes interconnectés
- Réduction drastique des erreurs techniques

Adaptabilité aux formats techniques spécialisés

- Traitement optimal par type de documentation technique
- Gestion native des schémas P&ID et diagrammes électriques
- Interprétation des tableaux de paramètres et spécifications
- Extensibilité à de nouveaux formats techniques

Capacité de raisonnement technique multi-étapes**



Traçabilité et explicabilité technique**



Évolutivité architecturale pour le secteur industriel**



1. Complexité d'implémentation technique

- Synchronisation des agents : Coordination complexe entre agents spécialisés pour éviter les conflits et les incohérences.
- Développement et maintenance : Besoin de pipelines d'orchestration avancés pour assurer un fonctionnement fluide.

2. Coûts computationnels

- Multiplication des appels aux modèles : Chaque agent peut nécessiter des requêtes spécifiques, augmentant les besoins en ressources.
- Infrastructure coûteuse : Nécessite des GPU/TPU puissants et une gestion optimisée des ressources pour éviter la surcharge.

3. Ingénierie système requise

- Intégration avec les systèmes existants : Adaptation nécessaire aux architectures industrielles souvent rigides et propriétaires.
- Sécurité et robustesse : Vérification des accès, gestion des droits et protection contre les biais ou erreurs d'agents autonomes.

CONCLUSION

Limites techniques

- Schémas techniques & PDF complexes
- Documentation évolutive
- Analyse comparative multi source
- Exigence réglementaire et conformité

Puissance des agents

- Précision et fiabilité technique accrues
- Adaptabilité aux formats industriels complexes
- Capacité de raisonnement technique avancé
- Intégration avec systèmes industriels

Nécessité d'une approche progressive adaptée au secteur

- Évaluation des besoins documentaires techniques
- Implémentation par phases selon criticité
- ROI mesurable à chaque étape
- Adaptation aux spécificités sectorielles